

AI-CARDS – AI-DRIVEN MOBILE APPLICATION FOR FANTASY CARD GENERATION

Candidate: Cristian-Ovidiu FAUR

Scientific Coordinator: Asist.dr.ing. Cosmin MARȘAVINA

3. USED TECHNOLOGIES

AI Cards is a mobile application that turns a free-form text prompt into a fantasy collectible card – a generated portrait, animated into a short video clip, accompanied by lore and combat statistics produced by a vision-language model. Building this experience required a wide set of technologies, ranging from the planning and version-control tools used during development, to the deep learning frameworks and pre-trained generative models that power the inference pipelines, and the cross-platform mobile runtime that delivers the result to the end user. This chapter covers, in turn, the tools used to plan and track the development process, the technologies behind the architecture and implementation of the back-end and front-end, and the local hosting environment on which the application runs.

3.1 PLANNING AND TRACKING THE DEVELOPMENT PROCESS

In the context of any non-trivial software project, managing time and the volume of work effectively is essential to a successful outcome. The development of AI Cards was planned around short, self-contained iterations – each iteration tackling a single vertical slice of the application (for example: “port the SDXL workflow from ComfyUI to a pure Python pipeline” or “add the gallery and history screens to the mobile client”). Source-code management, change tracking, and historical traceability were handled through Git and GitHub, while planning notes and per-feature design documents were kept directly inside the repository, under the `plans/` and `docs/` directories, so that they remain versioned alongside the code they describe.

3.1.1 GIT

Git is, in its essence, a distributed version-control system that helps with managing multiple revisions of a set of files. More concretely, it makes it possible to track the changes applied to a file and is particularly useful when, at some point, the developer wants to return to an earlier version of the file without having to undo the changes manually [1]. Throughout the development of AI Cards, Git was used not only as a safety net but also as a thinking tool: each meaningful refactor (for example, the migration from monolithic FastAPI handlers to a modular pipeline layout, or the redesign of the loading screen) was committed in isolation, so that the history could be read top-to-bottom as a narrative of how the application took shape.

3.1.2 GITHUB

For AI Cards, GitHub was used as the central remote that mirrored the local repository, as the platform on which pull requests were opened (`redesign`, `logo` and similar feature branches were merged through pull requests into `main`), and as the place in which the project history – including merged branches such as `redesign` and `logo` – can be browsed in retrospect.

3.2 ARCHITECTURE AND IMPLEMENTATION

AI Cards exposes a number of features designed to make the experience of a single user as smooth as possible: typing a prompt, watching the application produce a portrait, browsing previously generated cards in a gallery, and finally animating the chosen card into a short video. Behind the scenes, the architecture is split into three principal components, each implemented with a distinct set of technologies:

1. Front-end – the mobile interface:

- Implemented using the React Native framework with the Expo toolchain and TypeScript;
- Represents the part with which the user interacts when running the application on their phone;
- Is responsible for displaying the generated images, the card metadata and the resulting videos, as well as for orchestrating the user-facing loading animations while the back-end is running inference.

2. Back-end and application logic:

- Implemented using:
 - *FastAPI* – a modern Python web framework that exposes the REST endpoints consumed by the mobile client;
 - *PyTorch* and *HuggingFace Diffusers* – used to load and run the Stable Diffusion XL and Wan 2.2 models in pure Python, without depending on any external workflow runtime at inference time;
 - *Ollama* together with the *Qwen2.5-VL* vision-language model – used to caption the generated portrait and produce the card title, lore and combat statistics.
- Represents the part of the application that processes the prompts coming from the front-end, runs the AI pipelines on the local GPU, and exposes the resulting media back to the mobile client over HTTP.

3. Local storage and gallery:

- Implemented as a flat `output/` directory on the back-end host, with one PNG plus one JSON sidecar (`*_00001_.meta.json`) per generated card, and one MP4 per animated card;
- On the mobile client, generation history is mirrored locally through `AsyncStorage` so that previously generated cards can be re-opened without re-querying the server.

3.2.1 PYTHON

Python is the language in which the entire back-end – including the inference pipelines, the orchestration code and the REST API – is written. It was chosen because the dominant deep-learning ecosystem for inference on consumer GPUs (PyTorch, HuggingFace Transformers and Diffusers, safetensors, xformers) is Python-first, and because the FastAPI framework offers a very productive way to expose Python functions as type-checked HTTP endpoints. The `backend/` package targets Python 3.10 or newer in order to be able to use the union type syntax (`X | None`) as well as structural-pattern matching where convenient.

3.2.2 FASTAPI

FastAPI is a modern Python web framework for building HTTP APIs based on standard Python type hints. It uses Pydantic models to declare request and response payloads, and automatically validates the incoming data against those schemas. In AI Cards, FastAPI is used to expose four families of endpoints to the mobile client:

- `POST /generate/ex-image` – generates an image from a text prompt and returns both the image URL and the card metadata (title, lore, statistics, creativity score);
- `POST /generate/animate` – starts a video-generation job in the background and returns a `job_id`;
- `GET /generate/animate/status/{job_id}` – polls the progress of a video job;
- `GET /gallery`, `GET /gallery/{card_id}`, `DELETE /gallery/{card_id}` – list, fetch and remove previously generated cards;
- `GET /output/{filename}` – serves the generated PNG and MP4 files to the mobile client.

The application is hosted by an ASGI server – `uvicorn` – which is started locally with `uvicorn main:app -host 0.0.0.0 -port 8000`.

3.2.3 PYTORCH

PyTorch is the open-source deep-learning framework on top of which all the generative models used in this project are run. It provides the tensor library, the automatic differentiation engine and the CUDA bindings that allow the application to execute the SDXL and Wan 2.2 models on an NVIDIA GPU. Because the entire pipeline is purely an inference pipeline, automatic differentiation is disabled at run-time through `torch.inference_mode()`, which removes the bookkeeping needed for back-propagation and yields a small but consistent speed-up. Mixed-precision is used wherever possible: image generation runs in `float16`, while the Wan 2.2 weights are stored in the `float8_e4m3fn` (FP8) format and only up-cast to `float16` on the fly, in order to fit on a consumer GPU.

3.2.4 HUGGINGFACE DIFFUSERS

The `diffusers` library is a high-level abstraction layer over PyTorch that bundles together the most common building blocks of diffusion-based generative models – schedulers, UNets, VAEs, CLIP text encoders – behind a unified `Pipeline` interface. In `backend/pipelines/text_to_image.py`, the application loads the JuggernautXL Ragnarok checkpoint through `StableDiffusionXLPipeline.from_single_file`, swaps the default scheduler for an LCM scheduler in order to mirror the original ComfyUI workflow, and attaches the DMD2 4-step LoRA weights twice in sequence (once per “adapter pass”) so that the inference can converge in only eight denoising steps. The library also provides the helpers used to enable xformers memory-efficient attention, with a graceful fallback to PyTorch’s built-in attention slicing when xformers is not available.

3.2.5 STABLE DIFFUSION XL – JUGGERNAUTXL RAGNAROK

The text-to-image stage of the pipeline is built on top of Stable Diffusion XL (SDXL), specifically the community-trained *JuggernautXL Ragnarok* checkpoint. SDXL is a latent-diffusion model that pairs a UNet of roughly 2.6 billion parameters with two CLIP text encoders (CLIP-ViT-L and OpenCLIP-ViT-bigG), and a dedicated VAE that maps between the pixel space and the four-channel latent space in which diffusion takes place. JuggernautXL Ragnarok is a fine-tune of the base SDXL model that is biased toward photorealistic and cinematic portraits, which is well-suited for

the fantasy-card aesthetic that AI Cards targets. The default generation settings are 832×1216 pixels in portrait orientation, with a hard-coded negative prompt enforcing the avoidance of cartoon, deformed and anatomically broken outputs.

3.2.6 DMD2 LORA

Distribution Matching Distillation 2 (DMD2) is an acceleration technique for diffusion models, distributed as a set of LoRA weights that, when grafted onto an SDXL checkpoint, reduce the number of denoising steps required to obtain a sharp image from the typical 25–35 down to as few as 4–8. In the AI Cards text-to-image pipeline, the DMD2 SDXL 4-step LoRA is loaded twice as two separate adapter passes – exactly mirroring the ComfyUI workflow that served as a reference – and combined with the LCM scheduler so that inference converges in 8 steps with a low classifier-free guidance scale (CFG ≈ 1.4). This reduces image generation latency on a consumer GPU from tens of seconds to roughly two to three seconds per portrait.

3.2.7 WAN 2.2 IMAGE-TO-VIDEO

For the image-to-video stage, the application uses *Wan 2.2 I2V*, a diffusion-based model that takes an input image and a textual motion prompt, and produces a short video clip. Wan 2.2 is built around a dual-model design: two 14-billion-parameter UNets, both quantized to FP8, are used cooperatively during denoising – a *high-noise* model handles the early steps of the trajectory, where the latent is still mostly noise, and a *low-noise* model handles the late steps, where fine detail is reconstructed. Around these two UNets, the pipeline assembles a UMT5-XXL text encoder for conditioning on the prompt, and a Wan 2.1 VAE for encoding the start image into latent space and decoding the produced frames back into pixels.

The default generation settings are 640×640 pixels per frame, 81 frames per clip and Euler sampling. The text prompt at this stage describes the desired motion and atmosphere of the scene rather than its content – the visual content comes from the start image produced by the SDXL stage.

3.2.8 LIGHTX2V LORAS

LightX2V is a 4-step distillation technique for image-to-video diffusion models, conceptually similar to DMD2 but tailored for the temporal denoising trajectory. The AI Cards video pipeline loads two LightX2V LoRAs – one for the high-noise UNet and one for the low-noise UNet – and runs the denoising in two passes: a first pass that uses the high-noise model with the high-noise LoRA from step 0 up to a configurable

boundary step, and a second pass that uses the low-noise model with the low-noise LoRA from the boundary step up to the end. Together with the FP8 quantization of the underlying weights, these LoRAs are what makes it feasible to generate a 81-frame, 640×640 video clip on a single consumer-grade GPU within a reasonable amount of time.

3.2.9 OLLAMA AND QWEN2.5-VL

In addition to the visual pipelines, AI Cards enriches each generated card with a fantasy title, a short lore paragraph, and a set of combat statistics. This metadata is produced by a vision-language model running locally through *Ollama* – a self-hosted LLM runtime that exposes a unified HTTP API for downloading and querying open-weight models. The model used is *Qwen2.5-VL 7B*, which receives the generated portrait as a base64-encoded image alongside the user’s original prompt, and returns a strict JSON object containing the card’s title, lore, statistics, and a separate “creativity” score that judges the prompt itself rather than the image. The system prompt sent to Ollama enforces tone (Magic: The Gathering-style flavour text), length (under 50 words for the lore), and format (no Markdown, no extraneous keys), and a fall-back path is in place to return placeholder metadata if the local Ollama instance is unavailable.

3.2.10 COMFYUI AS A REFERENCE IMPLEMENTATION

ComfyUI is a node-based graphical interface for diffusion-based generative models. It was not used at runtime as part of the deployed application, but it served as the reference implementation against which the pure Python pipelines were validated: the JSON workflow files that ship with the repository (`Text2ImgWithLora.json`, `ImageToVideo.json`, `t2i.json`, `hires.json`) describe the exact node graph – checkpoint loader, CLIP text encode, KSampler, VAE decode – that the application reproduces in code. During development, every change to the inference pipelines was cross-checked against the corresponding ComfyUI workflow to confirm that the same prompt and seed produced visually equivalent results.

3.2.11 REACT NATIVE AND EXPO

The mobile client is built with *React Native*, a JavaScript framework for building native mobile applications, and *Expo*, a managed toolchain that sits on top of React Native and removes most of the friction of cross-platform mobile development. Expo provides a unified `npx expo start` development server that targets iOS, Android and the web simultaneously, a fully managed native build process (`expo run:android`, `expo run:ios`), and a curated collection of first-party modules. Among them, AI Cards relies on:

- **expo-router** – a file-based routing system inspired by Next.js, where each file under the `app/` directory becomes a route. The application uses it to declare its main screens (`home`, `loading`, `display`, `video-loading`, `gallery`, `history/[id]`, `battle-arena`, `battle-result`, `battle-select`, `battle`);
- **expo-av** – the audio/video module used to play back the MP4 clips returned by the back-end;
- **expo-file-system** – used to cache downloaded media on-device so that gallery entries can be opened without re-downloading;
- **expo-haptics** – triggers tactile feedback on key interactions, such as the moment a generation finishes;
- **expo-linear-gradient** – used heavily in the redesigned loading screens and the gilded card frames;
- **expo-image-manipulator** – used to crop and resize images before they are sent to the back-end for animation.

3.2.12 TYPESCRIPT

TypeScript is a statically typed superset of JavaScript that compiles to plain JavaScript. The entire mobile client is written in TypeScript, which means every screen, every component, every API call and every piece of persistent state is associated with an explicit type. This is particularly valuable at the boundary between the back-end and the front-end: the response shapes of the FastAPI endpoints (`ImageResponse`, `CardMeta`, `GalleryEntry`) are mirrored as TypeScript interfaces in `frontend/src/services/api.ts`, so any drift between the server and the client is caught at compile time by the `tsc -noEmit` type-check step rather than at runtime on the user's device.

3.2.13 REACT NATIVE REANIMATED AND SVG

The user-facing experience of AI Cards relies heavily on animation – in particular, the loading screens that play while the back-end is busy generating an image or a video clip. *React Native Reanimated* is the animation library used to drive these effects: it runs animations directly on the UI thread (rather than going through the JavaScript bridge on every frame), which keeps the loading screens fluid even when the device is otherwise busy. *React Native SVG* is used in tandem with Reanimated to draw the geometric loading patterns and the card-frame ornaments that surround the generated portraits.

3.2.14 ASYNCSTORAGE

AsyncStorage is the standard key-value persistence layer for React Native, with an asynchronous, promise-based API. AI Cards uses it to keep a local mirror of the user’s generation history, a per-day generation budget (the “generation currency” system declared in `frontend/src/storage/generationCurrency.ts`), and a small media cache index that maps remote URLs to locally cached files. This allows the user to re-open previously generated cards offline, and prevents the gallery screen from hammering the back-end with redundant requests every time it gains focus.

3.3 HOSTING THE APPLICATION

Because the inference pipelines used by AI Cards require a CUDA-capable NVIDIA GPU and several tens of gigabytes of model weights, the back-end is not deployed to a public cloud platform but is instead hosted on a local machine that the mobile client connects to over the local network. This section describes the runtime that exposes the FastAPI application, and the hardware profile under which the inference is run.

3.3.1 UVICORN

Uvicorn is a fast ASGI (Asynchronous Server Gateway Interface) web server, written in Python, that hosts FastAPI applications in production. It is invoked from the `backend/` directory with `uvicorn main:app --host 0.0.0.0 --port 8000`, where `0.0.0.0` makes the server reachable from any device on the same network as the host machine – in particular from the developer’s phone running the Expo client. Uvicorn handles the asynchronous request-response cycle and delegates the long-running, CPU-bound inference calls to a thread pool through FastAPI’s `run_in_threadpool` helper, so that the event loop remains responsive while a denoising pass is in flight.

3.3.2 LOCAL GPU INFERENCE

The inference workload runs on a local NVIDIA GPU. The model weights are stored on disk, and the application picks the correct path automatically based on the active operating system through the `get_model_profile` helper in `backend/config.py`. Memory is managed cooperatively across the two main pipelines: at any given time only one of the two Wan 2.2 UNets (high-noise or low-noise) holds the GPU's attention head, the other being either off-loaded to CPU memory or already evicted, in order to keep the peak GPU memory footprint compatible with consumer hardware.

BIBLIOGRAPHY

- [1] Git Project, *Git Documentation*. <https://git-scm.com/doc>.
- [2] GitHub, Inc., *GitHub Docs – About repositories*. <https://docs.github.com/en/repositories>.